

AgentMatrix 完整文档手册

多智能体动态协同与国产算力优化平台

版本: v0.1.0 | 日期: 2026-05-17

GitHub: <https://github.com/han1326189481/AgentMatrix>

第一部分：部署文档与安装说明

源代码获取

本项目源代码托管于 GitHub，请通过以下方式获取完整源代码：

克隆仓库

```
git clone https://github.com/han1326189481/AgentMatrix.git
cd AgentMatrix
```

GitHub 仓库地址

项目	地址
仓库首页	https://github.com/han1326189481/AgentMatrix
Git 克隆地址	<code>https://github.com/han1326189481/AgentMatrix.git</code>
SSH 克隆地址	<code>git@github.com:han1326189481/AgentMatrix.git</code>

提示: 建议使用 `git clone` 方式下载源代码，以确保获取完整的项目结构和版本历史。下载后请先阅读本文档完成环境配置和安装。

一、项目技术概况

1.1 开发语言

层级	语言	版本	框架/运行时
后端	Python	3.12+	FastAPI + Uvicorn
前端	TypeScript	5.3+	Next.js 14 + React 18
样式	CSS / TailwindCSS	3.4	PostCSS
脚本	PowerShell / Bash	-	启动/停止脚本

1.2 调用的模型清单

本地模型（通过 Ollama 运行）

模型名称	参数量	大小	用途
Qwen2.5 (<code>qwen2.5:1.5b</code>)	1.5B	~986 MB	Knowledge、Summary、Writer、Judge、Result Agent 的默认推理模型
Phi-4 Mini (<code>phi4-mini:3.8b</code>)	3.8B	~2.5 GB	Review Agent 专用推理模型（需要更强逻辑推理能力）

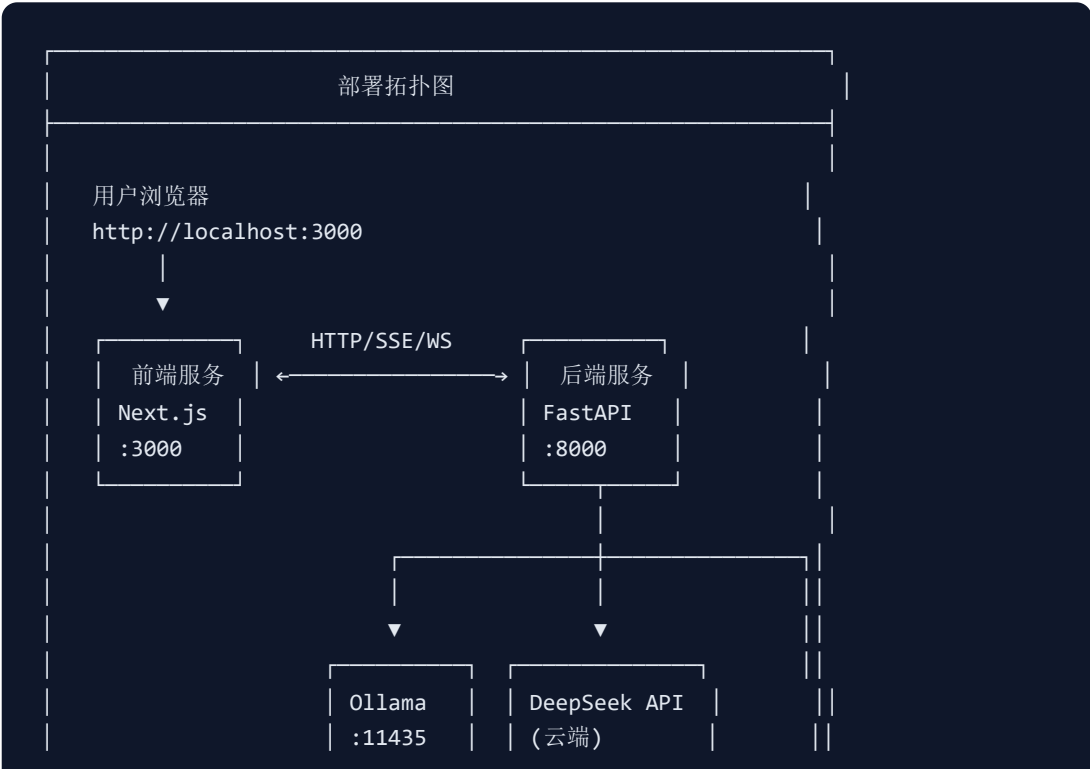
云端 API 模型

服务商	模型	用途
DeepSeek	<code>deepseek-r1-distill</code>	复杂任务云端增强推理（默认云端模型）

服务商	模型	用途
DeepSeek	deepseek-v4-flash	高速云端推理（可选）
DeepSeek	deepseek-chat	通用对话（可选）
DeepSeek	deepseek-r1	深度推理（可选）
OpenAI	gpt-4o / gpt-4-turbo / gpt-3.5-turbo	可选云端模型（需配置 API Key）
Anthropic	claude-3-5-sonnet / claude-3-opus / claude-3-sonnet	可选云端模型（需配置 API Key）
Google	gemini-pro / gemini-1.5-pro / gemini-1.5-flash	可选云端模型（需配置 API Key）

模型选择策略: Judge Agent 根据任务复杂度自动决定使用本地模型还是云端模型。复杂度 < 0.65 的任务由本地 Ollama 模型处理，复杂度 >= 0.65 的复杂任务自动升级至云端 DeepSeek API。云端模型可在前端「系统设置→添加云端模型」中自由切换。

二、部署架构概览





三、环境要求

3.1 硬件要求

环境	最低配置	推荐配置
开发环境	CPU 4核, 内存 8GB, 磁盘 20GB	CPU 8核, 内存 16GB, 磁盘 50GB
生产环境	CPU 4核, 内存 16GB, 磁盘 50GB	CPU 8核, 内存 32GB, 磁盘 100GB
GPU (可选)	-	NVIDIA GPU 8GB+ VRAM (加速本地模型推理)

3.2 软件要求

软件	最低版本	检查命令	用途
Python	3.12.0+	<code>python --version</code>	后端运行环境
Node.js	20.0.0+	<code>node --version</code>	前端运行环境
npm	9.0.0+	<code>npm --version</code>	前端包管理
Ollama	0.23.0+	<code>ollama --version</code>	本地模型推理服务
Git	2.40.0+	<code>git --version</code>	代码版本管理

3.3 操作系统支持

操作系统	支持状态	架构	备注
Windows 10/11	✅ 完全支持	x86_64	PowerShell 5.1+

操作系统	支持状态	架构	备注
macOS 12+	完全支持	x86_64 / ARM64	原生终端
Ubuntu 22.04+	完全支持	x86_64 / ARM64	Bash, apt 包管理
统信UOS (桌面专业版)	完全支持	x86_64 / ARM64 (鲲鹏)	基于 Deepin, apt 包管理
统信UOS (服务器版)	完全支持	x86_64 / ARM64 (鲲鹏)	基于 Deepin, apt 包管理
麒麟OS (桌面版 V10)	完全支持	x86_64 / ARM64 (鲲鹏/飞腾)	基于 openEuler/Ubuntu, rpm 或 apt
麒麟OS (服务器版 V10)	完全支持	x86_64 / ARM64 (鲲鹏/飞腾)	基于 openEuler, rpm 包管理
CentOS 7 / RHEL 7+	支持	x86_64	yum/dnf 包管理

说明: 统信UOS 和麒麟OS 均为国产自主可控操作系统，本项目已完成兼容性适配。部署方式与 Linux 基本一致，仅在包管理器和基础环境安装方式上有细微差异，详见下方各系统的安装指引。

四、安装基础环境

4.1 安装 Python 3.12+

Windows: 1. 访问 <https://www.python.org/downloads/> 2. 下载 Python 3.12 安装包 3. 安装时勾选 "Add Python to PATH" 4. 验证: `python --version`

Ubuntu / 统信UOS (桌面版):

```
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
sudo apt install python3.12 python3.12-venv
```

统信UOS (服务器版) / 麒麟OS (桌面版 V10):

```
# 统信UOS 和麒麟OS 桌面版通常自带 Python 3.x
# 若版本不满足，可从源码编译安装
sudo apt update
sudo apt install build-essential libssl-dev zlib1g-dev libbz2-dev libreadline-
dev libsqlite3-dev

# 下载 Python 3.12 源码
wget https://www.python.org/ftp/python/3.12.0/Python-3.12.0.tgz
tar -xzf Python-3.12.0.tgz
cd Python-3.12.0
./configure --enable-optimizations
make -j$(nproc)
sudo make altinstall

# 验证
python3.12 --version
```

麒麟OS (服务器版 V10) / CentOS / RHEL:

```
# 安装编译依赖
sudo dnf install -y gcc openssl-devel bzip2-devel libffi-devel zlib-devel read
line-devel sqlite-devel

# 下载 Python 3.12 源码
wget https://www.python.org/ftp/python/3.12.0/Python-3.12.0.tgz
tar -xzf Python-3.12.0.tgz
cd Python-3.12.0
./configure --enable-optimizations
make -j$(nproc)
sudo make altinstall

# 验证
python3.12 --version
```

macOS:

```
brew install python@3.12
```

4.2 安装 Node.js 20+

Windows / macOS: 1. 访问 <https://nodejs.org/> 2. 下载 Node.js 20 LTS 版本 3. 验证: `node -`
`-version` 和 `npm --version`

Ubuntu / 统信UOS (桌面版):

```
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -
```

```
sudo apt install nodejs
```

统信UOS (服务器版) / 麒麟OS (桌面版 V10):

```
# 方式一: 使用 NodeSource (apt)
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -
sudo apt install -y nodejs

# 方式二: 使用 nvm (推荐, 更灵活)
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.39.7/install.sh | bash
source ~/.bashrc
nvm install 20
nvm use 20
```

麒麟OS (服务器版 V10) / CentOS / RHEL:

```
# 使用 nvm 安装 (推荐)
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.39.7/install.sh | bash
source ~/.bashrc
nvm install 20
nvm use 20

# 或使用 NodeSource (dnf)
curl -fsSL https://rpm.nodesource.com/setup_20.x | sudo bash -
sudo dnf install -y nodejs
```

4.3 安装 Ollama

Windows: 1. 访问 <https://ollama.com/download/windows> 2. 下载并安装 3. 验证: `ollama -version`

Ubuntu / 统信UOS / 麒麟OS (基于 apt 的系统):

```
curl -fsSL https://ollama.com/install.sh | sh
```

麒麟OS (基于 rpm 的系统) / CentOS / RHEL:

由于 Ollama 官方暂未提供 rpm 包, 建议使用以下两种方式之一:

```
# 方式一: 使用官方 Linux 安装脚本 (通用性最好)
curl -fsSL https://ollama.com/install.sh | sh

# 方式二: 手动下载二进制文件
# 下载适用于 Linux 的 Ollama 二进制文件
wget https://ollama.com/download/ollama-linux-amd64.tgz
```

```
sudo tar -C /usr -xzf ollama-linux-amd64.tgz

# 创建 systemd 服务
sudo tee /etc/systemd/system/ollama.service > /dev/null << 'EOF'
[Unit]
Description=Ollama Service
After=network.target

[Service]
Type=simple
User=root
ExecStart=/usr/bin/ollama serve
Restart=always
Environment="OLLAMA_HOST=0.0.0.0:11435"

[Install]
WantedBy=multi-user.target
EOF

sudo systemctl daemon-reload
sudo systemctl enable ollama
sudo systemctl start ollama
```

注意: 在国产 ARM64 架构（鲲鹏 920、飞腾 S2500 等）上部署时，Ollama 和模型均提供原生 ARM64 支持，安装命令与 x86_64 一致。

macOS:

```
brew install ollama
```

五、安装步骤

5.1 获取源代码

```
# 从 GitHub 克隆项目
git clone https://github.com/han1326189481/AgentMatrix.git
cd AgentMatrix
```

5.2 配置 Ollama 并下载本地模型

Ollama 默认监听 11434 端口。本项目的默认配置使用 **11435** 端口（避免端口冲突）：

```
# Linux / macOS / 统信UOS / 麒麟OS
export OLLAMA_HOST="0.0.0.0:11435"
ollama serve

# Windows PowerShell
$env:OLLAMA_HOST="0.0.0.0:11435"
ollama serve
```

下载所需的本地模型（各约需 5-20 分钟）：

```
# 下载 Qwen2.5 1.5B 模型（~986MB）
ollama pull qwen2.5:1.5b

# 下载 Phi-4 Mini 3.8B 模型（~2.5GB）
ollama pull phi4-mini:3.8b

# 验证模型已安装
ollama list
```

预期输出：

NAME	ID	SIZE	MODIFIED
phi4-mini:3.8b	78fad5d182a7	2.5 GB	...
qwen2.5:1.5b	65ec06548149	986 MB	...

验证 Ollama 服务：

```
curl http://localhost:11435/api/tags
```

5.3 后端安装

Windows

```
cd backend

# 创建虚拟环境
python -m venv venv

# 激活虚拟环境（如果遇到执行策略限制，先运行：Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser）
.\venv\Scripts\Activate.ps1
```

```
# 安装核心依赖
pip install fastapi==0.110.0 uvicorn==0.29.0 pydantic==2.6.0
pip install python-dotenv==1.0.0 aiohttp==3.9.0 sqlalchemy==2.0.0
pip install loguru==0.7.0 websockets==12.0 psutil==5.9.0
pip install python-pptx==0.6.23 python-docx==1.1.0 pymdown-extensions==10.0.0

cd ..
```

Ubuntu / 统信UOS (桌面版) / macOS

```
cd backend

# 创建虚拟环境
python3 -m venv venv
source venv/bin/activate

# 安装依赖
pip install fastapi uvicorn pydantic python-dotenv aiohttp sqlalchemy loguru w
ebsockets psutil
pip install python-pptx python-docx pymdown-extensions

cd ..
```

统信UOS (服务器版) / 麒麟OS (桌面版 V10)

```
cd backend

# 创建虚拟环境（注意：部分国产OS的 Python 命令可能为 python3.12）
python3.12 -m venv venv
source venv/bin/activate

# 安装依赖
pip install fastapi uvicorn pydantic python-dotenv aiohttp sqlalchemy loguru w
ebsockets psutil
pip install python-pptx python-docx pymdown-extensions

cd ..
```

麒麟OS (服务器版 V10) / CentOS / RHEL

```
cd backend

# 使用 altinstall 安装的 Python 3.12
python3.12 -m venv venv
source venv/bin/activate
```

```
# 安装依赖
pip install fastapi uvicorn pydantic python-dotenv aiohttp sqlalchemy loguru websockets psutil
pip install python-pptx python-docx pymdown-extensions

cd ..
```

5.4 前端安装

所有操作系统:

```
cd frontend
npm install
cd ..
```

5.5 配置环境变量

后端配置

创建 `backend/.env` 文件:

Windows PowerShell:

```
@
SERVER_HOST=0.0.0.0
SERVER_PORT=8000
SERVER_RELOAD=true
LOG_LEVEL=INFO
DATABASE_URL=sqlite:///./agentmatrix.db
OLLAMA_HOST=http://localhost:11435
OLLAMA_MODEL=qwen2.5:1.5b
OLLAMA_REVIEW_MODEL=phi4-mini:3.8b
DEEPSEEK_API_KEY=
DEEPSEEK_API_BASE=https://api.deepseek.com/v1
DEEPSEEK_MODEL=deepseek-r1-distill
COMPLEXITY_THRESHOLD=0.65
ALLOWED_ORIGINS=http://localhost:3000,http://localhost:8000
"@ | Out-File -FilePath backend\.env -Encoding UTF8
```

Linux / macOS / 统信UOS / 麒麟OS:

```
cat > backend/.env << 'EOF'
SERVER_HOST=0.0.0.0
```

```
SERVER_PORT=8000
OLLAMA_HOST=http://localhost:11434
OLLAMA_MODEL=qwen2.5:1.5b
OLLAMA_REVIEW_MODEL=phi4-mini:3.8b
DEEPSEEK_API_KEY=
DEEPSEEK_API_BASE=https://api.deepseek.com/v1
DEEPSEEK_MODEL=deepseek-r1-distill
COMPLEXITY_THRESHOLD=0.65
ALLOWED_ORIGINS=http://localhost:3000,http://localhost:8000
EOF
```

说明: DeepSeek API Key 为可选项。不配置则所有任务由本地 Ollama 模型处理；配置后复杂任务将自动调用云端 DeepSeek 增强推理。

国产 OS 注意: 如果在服务器环境中部署，建议将 `ALLOWED_ORIGINS` 设置为实际的访问域名或 IP。

前端配置

创建 `frontend/.env.local` 文件：

```
NEXT_PUBLIC_API_URL=http://localhost:8000
NEXT_PUBLIC_WS_URL=ws://localhost:8000
```

六、启动服务

6.1 开发环境启动

需要同时启动三个终端窗口：

终端 1 - Ollama 服务:

```
ollama serve
```

终端 2 - 后端服务:

Windows:

```
cd backend
.\venv\Scripts\Activate.ps1
```

```
uvicorn app.main:app --host 0.0.0.0 --port 8000 --reload
```

Linux / macOS / 统信UOS / 麒麟OS:

```
cd backend
source venv/bin/activate
uvicorn app.main:app --host 0.0.0.0 --port 8000 --reload
```

启动成功日志:

```
INFO:      Database tables created successfully
INFO:      Database initialized successfully
INFO:      Loaded knowledge base with 20 keywords
INFO:      All agents initialized successfully
INFO:      WebSocket manager initialized
INFO:      Application startup complete.
```

终端 3 - 前端服务:

```
cd frontend
npm run dev
```

启动成功日志:

```
▲ Next.js 14.1.0
- Local:      http://localhost:3000
✓ Ready in 5.4s
```

6.2 Windows 一键启动

```
# 在项目根目录运行
.\start.ps1
```

七、安装验证

7.1 验证 Ollama

```
ollama list
```

7.2 验证后端

```
curl http://localhost:8000/health
# 预期: {"status":"healthy","agents":{"knowledge":{"status":"ready"},...}}
```

浏览器访问 <http://localhost:8000/docs> 查看 API 文档。

7.3 验证前端

浏览器访问 <http://localhost:3000>, 预期看到 AgentMatrix 主界面: - ☒ 顶部任务输入框 - ☒ 左侧 Agent 舰队列表 - ☒ 中间流水线可视化 - ☒ 右侧输出面板

7.4 端到端测试

在输入框输入 "你好, 请介绍一下你自己", 预期: 6 个 Agent 依次执行, 左侧"成本节省"指标增加, 右侧显示最终结果。

八、生产环境部署

8.1 生产模式启动

后端:

```
cd backend
uvicorn app.main:app --host 0.0.0.0 --port 8000 --workers 4
```

前端:

```
cd frontend
npm run build
npm start
```

8.2 使用 systemd (Linux / 统信UOS / 麒麟OS)

后端服务 (`/etc/systemd/system/agentmatrix-backend.service`):

```
[Unit]
Description=AgentMatrix Backend Service
```

```

After=network.target ollama.service

[Service]
Type=simple
User=agentmatrix
WorkingDirectory=/opt/agentmatrix/backend
Environment=PATH=/opt/agentmatrix/backend/venv/bin:/usr/bin
ExecStart=/opt/agentmatrix/backend/venv/bin/uvicorn app.main:app --host 0.0.0.0 --port 8000 --workers 4
Restart=always
RestartSec=5

[Install]
WantedBy=multi-user.target

```

前端服务 (`/etc/systemd/system/agentmatrix-frontend.service`):

```

[Unit]
Description=AgentMatrix Frontend Service
After=network.target agentmatrix-backend.service

[Service]
Type=simple
User=agentmatrix
WorkingDirectory=/opt/agentmatrix/frontend
ExecStart=/usr/bin/npm start
Restart=always
RestartSec=5

[Install]
WantedBy=multi-user.target

```

```

sudo systemctl daemon-reload
sudo systemctl enable agentmatrix-backend agentmatrix-frontend
sudo systemctl start agentmatrix-backend agentmatrix-frontend

```

8.3 国产 OS 生产部署注意事项

统信UOS 服务器版

```

# 1. 确保防火墙开放端口
sudo ufw allow 3000/tcp
sudo ufw allow 8000/tcp

# 2. 若使用 SELinux (默认启用)
sudo semanage port -a -t http_port_t -p tcp 3000
sudo semanage port -a -t http_port_t -p tcp 8000

```

```
# 3. 设置为开机自启动
sudo systemctl enable ollama agentmatrix-backend agentmatrix-frontend
```

麒麟OS 服务器版 V10

```
# 1. 麒麟OS 使用 firewalld 管理防火墙
sudo firewall-cmd --permanent --add-port=3000/tcp
sudo firewall-cmd --permanent --add-port=8000/tcp
sudo firewall-cmd --reload

# 2. 若使用 SELinux
sudo semanage port -a -t http_port_t -p tcp 3000
sudo semanage port -a -t http_port_t -p tcp 8000

# 3. 设置开机自启
sudo systemctl enable ollama agentmatrix-backend agentmatrix-frontend
```

ARM64 架构 (鲲鹏 920 / 飞腾 S2500) 特殊说明

```
# 国产 ARM64 CPU 上部署时注意:
# 1. Python 建议从源码编译以充分利用 ARM 优化
# 2. Ollama 原生支持 ARM64, 直接使用官方脚本安装
# 3. Node.js 提供官方 ARM64 二进制包
# 验证架构
uname -m # 应输出: aarch64
```

8.4 使用 Docker

docker-compose.yml:

```
version: '3.8'

services:
  backend:
    build: ./backend
    ports:
      - "8000:8000"
    environment:
      - OLLAMA_HOST=http://ollama:11434
      - DEEPSEEK_API_KEY=${DEEPSEEK_API_KEY}
    volumes:
      - ./backend/data:/app/data
    depends_on:
      - ollama
    restart: unless-stopped
```

```

frontend:
  build: ./frontend
  ports:
    - "3000:3000"
  environment:
    - NEXT_PUBLIC_API_URL=http://backend:8000
    - NEXT_PUBLIC_WS_URL=ws://backend:8000
  depends_on:
    - backend
  restart: unless-stopped

ollama:
  image: ollama/ollama:latest
  ports:
    - "11435:11434"
  volumes:
    - ollama_data:/root/.ollama
  restart: unless-stopped

volumes:
  ollama_data:

```

```

# 启动全部服务
docker-compose up -d

# 下载模型到容器
docker-compose exec ollama ollama pull qwen2.5:1.5b
docker-compose exec ollama ollama pull phi4-mini:3.8b

```

8.5 Nginx 反向代理配置

```

server {
    listen 80;
    server_name agentmatrix.example.com;

    location / {
        proxy_pass http://localhost:3000;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection "upgrade";
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }

    location /api/ {
        proxy_pass http://localhost:8000/api/;
        proxy_http_version 1.1;
        proxy_set_header Host $host;
    }
}

```

```
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    }

    location /ws {
        proxy_pass http://localhost:8000/ws;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection "upgrade";
        proxy_set_header Host $host;
    }
}
```

九、健康检查与监控

9.1 健康检查端点

端点	方法	用途
/health	GET	系统整体健康状态 + Agent 状态
/api/v1/metrics	GET	系统运行指标
/api/v1/chat/health	GET	聊天服务状态
/api/v1/workflow/cache/stats	GET	工作流缓存统计

9.2 系统监控指标

GET /api/v1/metrics 返回: - total_requests - 总请求数 - local_executions - 本地执行次数 - cloud_executions - 云端执行次数 - api_calls - API 调用次数 - cost_saved - 节省费用估算

9.3 日志配置

```
# backend/.env
LOG_LEVEL=INFO          # 开发: DEBUG, 生产: INFO 或 WARNING
LOG_FILE=logs/system.log
```

十、常见问题排查

Q1: PowerShell 无法激活虚拟环境?

错误: "无法加载文件 .\venv\Scripts\Activate.ps1, 因为在此系统上禁止运行脚本。"
解决: 以管理员身份运行 PowerShell, 执行:
`Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser`

Q2: pip install 报错?

解决:
1. 确保 Python ≥ 3.12 : `python --version`
2. 更新 pip: `python -m pip install --upgrade pip`
3. 如果在公司网络, 配置代理: `pip install --proxy=http://proxy:port ...`

Q3: npm install 报错?

解决:
1. 删除 `node_modules` 和 `package-lock.json`
2. 重新安装: `npm install`
3. 或使用: `npm install --legacy-peer-deps`

Q4: Ollama 连接失败?

现象: 后端日志 "Failed to call Ollama: Connection refused"
解决:
1. 确认 Ollama 正在运行: `ollama list`
2. 检查端口号: Windows 项目配置 11435, Linux/macOS/国产OS 默认 11434
3. 测试连接: `curl http://localhost:11435/api/tags`
4. 修改 `backend/.env` 中的 `OLLAMA_HOST` 为正确端口

Q5: 前端页面空白?

解决:
1. 确认后端正在运行: `curl http://localhost:8000/health`
2. 检查 `frontend/.env.local` 配置
3. 打开浏览器开发者工具 (F12) 查看错误信息

Q6: 端口被占用?

现象: uvicorn 启动报 "Address already in use"

解决:

1. 查找占用进程: `netstat -ano | findstr :8000` (Windows) / `ss -tlnp | grep 8000` (Linux)
2. 修改端口: 在 `.env` 中设置 `SERVER_PORT=8001`

Q7: DeepSeek API 调用失败?

现象: "Error: DeepSeek API Key 未设置"

解决:

1. 访问 <https://platform.deepseek.com/> 获取 API Key
2. 添加到 `backend/.env` 中 `DEEPSEEK_API_KEY`
3. 不配置 API Key 则所有任务本地处理

Q8: 模型下载速度慢?

解决:

1. 使用代理加速下载
2. 仅下载需要的模型 (至少需要 `qwen2.5:1.5b`)
3. 可在另一台机器下载后拷贝模型文件

Q9: 统信UOS / 麒麟OS 上 Python 命令找不到?

现象: "python3: command not found"

解决:

```
# 统信UOS / 麒麟OS 桌面版可能使用 python3 别名
which python3 || which python3.12
```

```
# 创建软链接 (如果需要)
```

```
sudo ln -s /usr/bin/python3.12 /usr/bin/python3
```

Q10: 国产 ARM64 平台上安装依赖失败?

现象: pip 安装某些包时编译失败

解决:

1. 确保安装了 gcc 和开发库: `sudo apt install build-essential python3-dev`
2. 某些包可能尚未提供 ARM64 预编译 wheel, pip 会自动从源码编译
3. 如遇到特定包编译失败, 可尝试使用系统包管理器安装对应的 `-dev` 包

十一、安全注意事项

- 1. **API 密钥管理:** 不要将 `.env` 文件提交到版本控制。已加入 `.gitignore`。
- 2. **生产环境 CORS:** 修改 `ALLOWED_ORIGINS` 为实际前端域名，不要使用 `*`。
- 3. **防火墙:** 生产环境应配置防火墙，仅开放必要端口（80/443）。
- 4. **HTTPS:** 生产环境应使用 SSL/TLS 加密通信。
- 5. **定期更新:** 定期更新 Ollama 模型和后端依赖，获取安全补丁。
- 6. **国产 OS 安全:** 统信UOS 和麒麟OS 默认启用安全策略（SELinux / AppArmor），部署时如遇到权限问题，请检查安全日志并配置相应策略。

十二、目录结构说明

```
AgentMatrix/
├── backend/                                # 后端服务 (Python + FastAPI)
│   ├── agents/                            # 6 个 Agent 实现
│   │   ├── base/                          # Agent 基类和注册中心
│   │   ├── knowledge/                     # Knowledge Agent (知识检索)
│   │   ├── summary/                       # Summary Agent (需求摘要)
│   │   ├── writer/                       # Writer Agent (内容生成)
│   │   ├── review/                       # Review Agent (质量评审)
│   │   ├── judge/                        # Judge Agent (复杂度判断)
│   │   └── result/                        # Result Agent (成果导出)
│   ├── api/                              # API 路由
│   │   ├── v1/                           # v1 版本 API
│   │   └── websocket/                     # WebSocket 管理
│   ├── core/                              # 核心业务逻辑
│   │   ├── workflow/                     # 工作流编排
│   │   ├── llm/                          # LLM 客户端
│   │   └── dynamic_router/                # 动态算力路由
│   ├── models/                           # Pydantic 数据模型
│   ├── prompts/                           # Prompt 模板
│   ├── knowledge/                         # 知识库服务
│   ├── config/                           # 配置管理
│   ├── services/                         # 聚合服务层
│   ├── app/                              # 应用入口
│   │   ├── main.py                       # FastAPI 入口
│   │   ├── config.py                     # 全局配置
│   │   └── database.py                   # 数据库初始化
│   └── tests/                             # 测试
├── frontend/                              # 前端服务 (TypeScript + Next.js)
│   ├── src/
│   │   └── app/                           # Next.js App Router
```

```
| | | components/      # React 组件
| | |   └─ layout/      # 主界面布局
| | |   └─ stores/       # Zustand 状态管理
| | |   └─ services/api/ # API 调用封装
| | |   └─ types/        # TypeScript 类型定义
| | └─ package.json     # 依赖清单
| └─ next.config.js     # Next.js 配置
|
└─ configs/            # 配置文件
└─ docs/               # 项目文档
└─ scripts/            # 辅助脚本
└─ start.ps1           # Windows 一键启动脚本
└─ start.bat           # Windows 批处理启动脚本
└─ stop.ps1            # Windows 停止脚本
└─ README.md           # 项目说明
```

第二部分：技术报告

一、项目概述

AgentMatrix 是一个基于多智能体协同架构与动态算力路由的 AI 应用平台。系统通过六个专业化 Agent (Knowledge、Summary、Writer、Review、Judge、Result) 的流水线协作，结合 Judge Agent 的复杂度感知机制，实现任务的智能分流：简单任务由本地 Ollama 模型 (Qwen2.5:1.5b、Phi4-mini:3.8b) 高效处理，复杂任务自动升级至云端 DeepSeek 等大模型进行增强推理，从而在保障输出质量的同时显著降低 API 调用成本。

1.1 开发语言与框架

层级	语言	版本	框架 / 运行时
后端	Python	3.12+	FastAPI 0.110+ + Uvicorn 0.29+
前端	TypeScript	5.3+	Next.js 14 + React 18
前端样式	CSS / TailwindCSS	3.4	PostCSS
前端状态管理	TypeScript	-	Zustand 4.5

层级	语言	版本	框架 / 运行时
前端流程图	TypeScript	-	ReactFlow 11.10
前端图表	TypeScript	-	Chart.js 4.4 + react-chartjs-2
前端动画	TypeScript	-	Framer Motion 10.18
数据校验	Python	-	Pydantic 2.6+
数据库ORM	Python	-	SQLAlchemy 2.0+
脚本部署	PowerShell / Bash	-	启动/停止/一键部署

1.2 调用的模型清单

本地模型 (通过 Ollama 运行)

模型名称	参数量	大小	推理引擎	用途
Qwen2.5 (qwen2.5:1.5b)	1.5B	~986 MB	Ollama	Knowledge、Summary、Writer、Judge、Result Agent 的默认推理模型
Phi-4 Mini (phi4-mini:3.8b)	3.8B	~2.5 GB	Ollama	Review Agent 专用推理模型 (需要更强的逻辑推理和评分能力)

云端 API 模型 (可选配置)

服务商	模型标识	类型	用途
DeepSeek	deepseek-r1-distill	推理增强	复杂任务云端增强推理 (默认云端模型)
DeepSeek	deepseek-v4-flash	高速推理	轻量级云端高速推理
DeepSeek	deepseek-chat	通用对话	通用对话与文本生成
DeepSeek	deepseek-r1	深度推理	高复杂度深度推理任务

服务商	模型标识	类型	用途
OpenAI	gpt-4o / gpt-4-turbo / gpt-3.5-turbo	通用	可选云端模型（需配置 API Key）
Anthropic	claude-3-5-sonnet / claude-3-opus / claude-3-sonnet	通用	可选云端模型（需配置 API Key）
Google	gemini-pro / gemini-1.5-pro / gemini-1.5-flash	通用	可选云端模型（需配置 API Key）

模型选择策略: Judge Agent 根据任务复杂度自动决定使用本地模型还是云端模型。复杂度 < 0.65 的任务由本地 Ollama 模型处理，复杂度 >= 0.65 的复杂任务自动升级至云端 API。云端模型可在前端「系统设置→添加云端模型」中自由切换服务商和模型。

1.3 操作系统兼容性

操作系统	支持状态	架构
Windows 10/11	✅ 完全支持	x86_64
macOS 12+	✅ 完全支持	x86_64 / ARM64 (Apple Silicon)
Ubuntu 22.04+	✅ 完全支持	x86_64 / ARM64
统信UOS (桌面专业版 / 服务器版)	✅ 完全支持	x86_64 / ARM64 (鲲鹏920)
麒麟OS (桌面版 / 服务器版 V10)	✅ 完全支持	x86_64 / ARM64 (鲲鹏920 / 飞腾S2500)
CentOS 7 / RHEL 7+	✅ 支持	x86_64

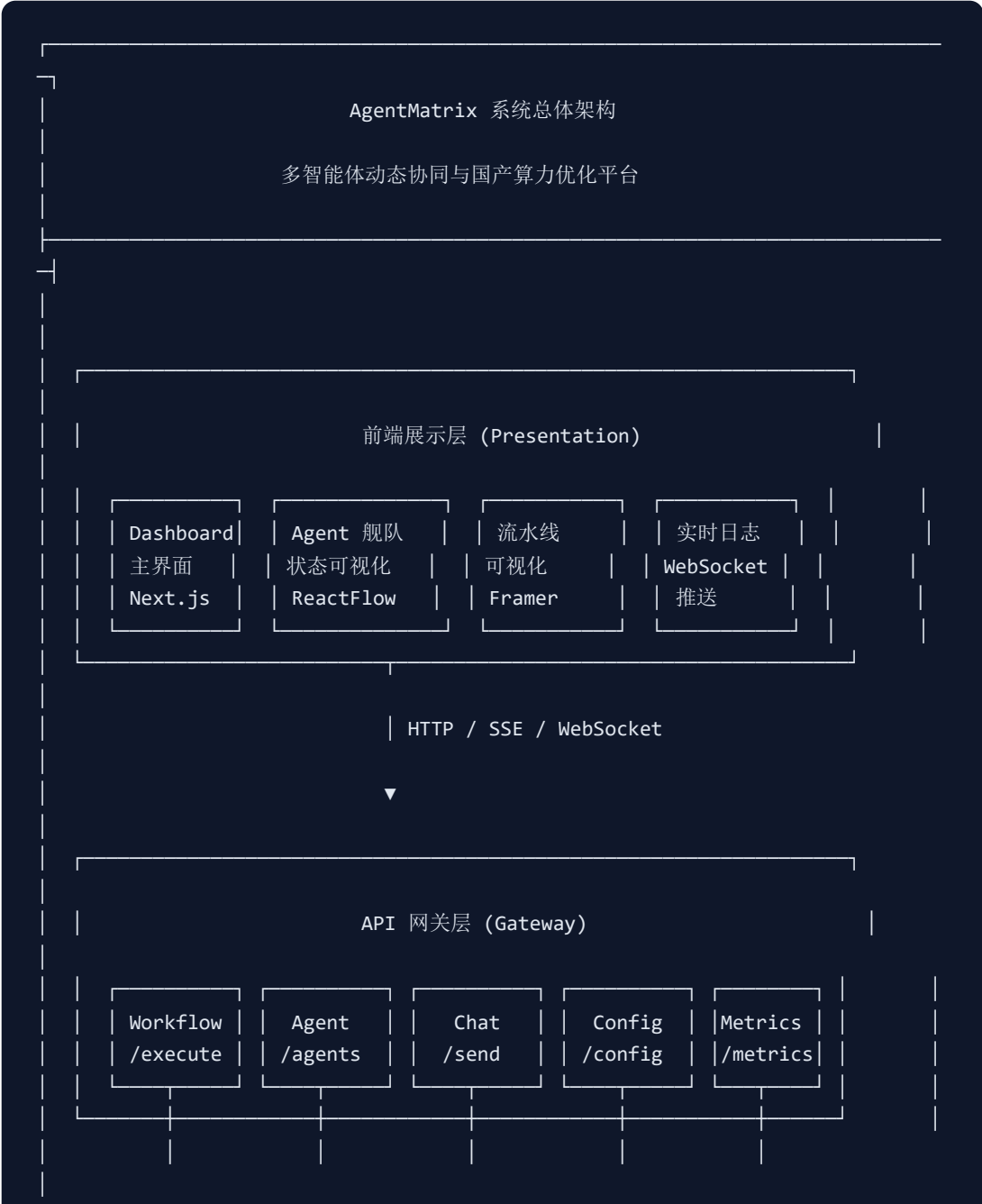
1.4 核心设计目标

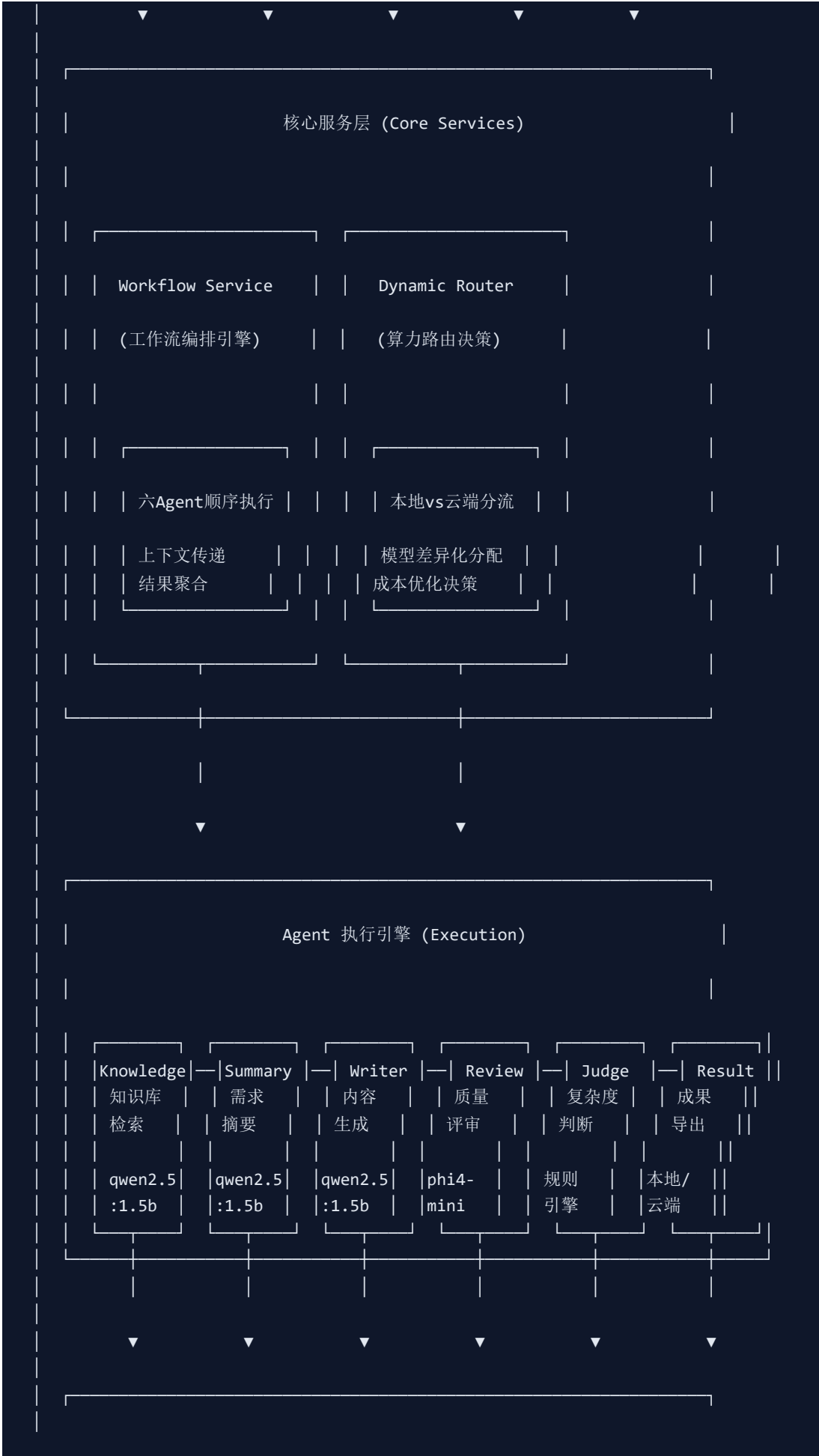
目标	实现策略
成本优化	80%+ 简单任务本地执行，仅复杂任务调用云端 API
响应速度	本地模型毫秒级响应，避免网络延迟
质量保障	云端大模型兜底复杂场景，Review Agent 质量评审
国产算力	优先使用 Qwen2.5、DeepSeek 等国产模型，支持鲲鹏/飞腾 ARM64

目标	实现策略
可扩展性	插件化 Agent 架构，支持模型热切换，多服务商适配
自主可控	全栈支持统信UOS、麒麟OS等国产操作系统

二、系统架构

2.1 总体架构图







2.2 分层架构说明

层级	核心组件	技术栈	职责
展示层	DashboardLayout, AgentFleet, PipelineFlow	Next.js 14, React 18, TailwindCSS 3.4, Zustand 4.5, Framer Motion 10.18, ReactFlow 11.10, Chart.js 4.4	用户交互界面、多 Agent 状态可视化、流水线动画、实时数据推送
网关层	API v1 Router, WebSocket Manager	FastAPI, Uvicorn, websockets 12.0+, SSE	请求路由与分发、参数校验 (Pydantic)、流式响应 (SSE)、WebSocket

层级	核心组件	技术栈	职责
			双向通信、CORS 管理
服务层	WorkflowService, DynamicRouter, AgentService	Python asyncio, SimpleCache	六Agent工作流编排、算力路由决策、模型差异化分配、缓存策略管理
执行层	Six Agents (Knowledge→Summary→Writer→Review→Judge→Result)	规则引擎 + LLM 推理, PromptManager	知识检索→需求摘要→内容生成→质量评审→复杂度判断→成果导出
模型层	LLMClient, DeepSeekClient	Ollama API, DeepSeek API v1, OpenAI-compatible	本地 Ollama 模型调用、云端多服务商 API 统一适配、流式/非流式双模式
数据层	SQLite + JSON + SimpleCache + Prompt Templates	SQLAlchemy 2.0+, JSON, markdown	持久化存储、知识库管理（7分类/20关键词）、Prompt 模板渲染、三层缓存

2.3 六智能体协同流水线



三种通信方式协同工作			
通信方式	HTTP/REST	SSE (流式)	WebSocket
用途	CRUD 操作	实时输出流	状态推送
典型场景	Agent状态查询	聊天流式响应	Agent状态
端点	/api/v1/*	/chat/send/stream	/ws
数据方向	请求 ↔ 响应	后端 → 前端	双向通信
传输协议	HTTP/1.1	HTTP/1.1	WebSocket
前端实现	Axios	fetch + Reader	Socket.IO

3.1 设计理念

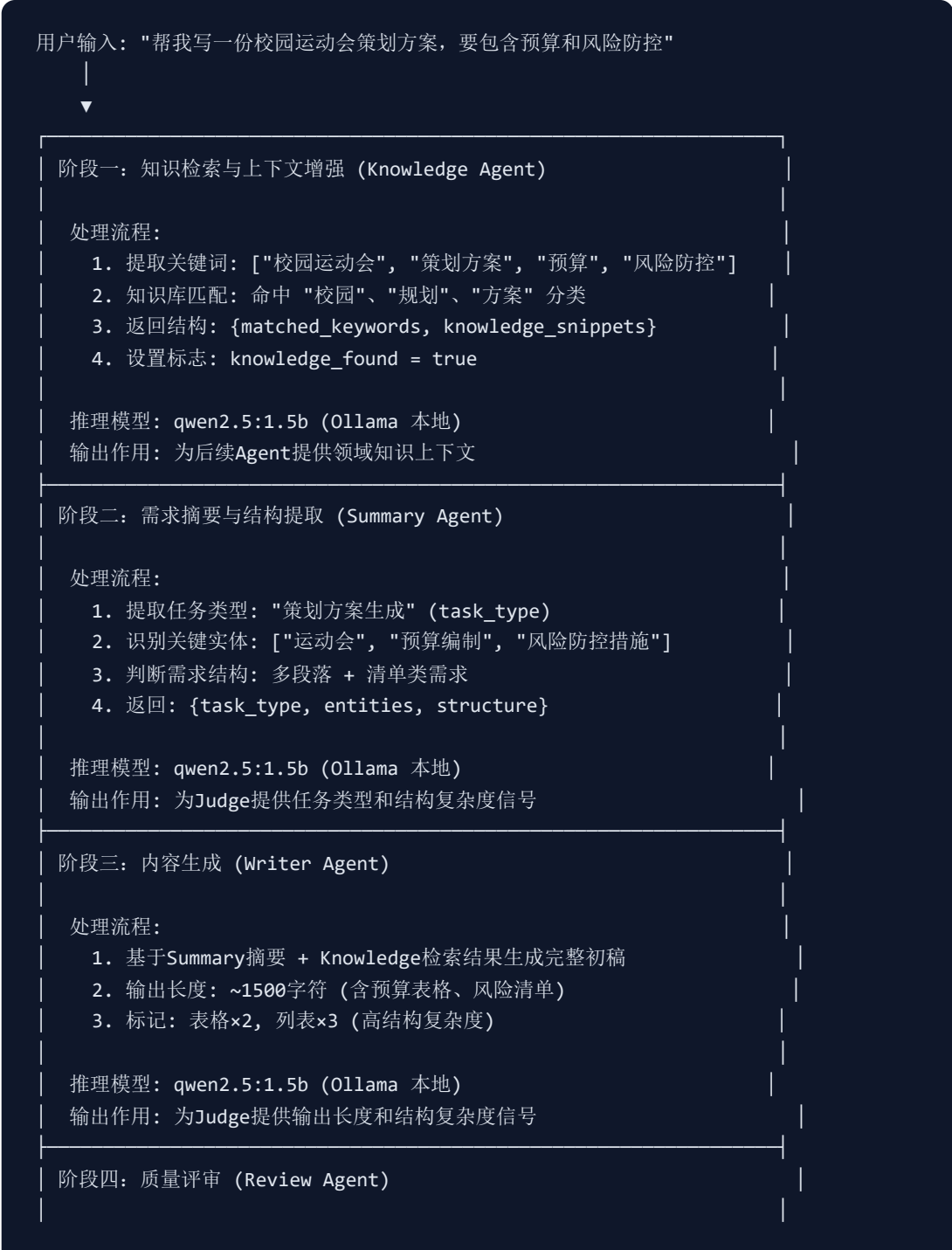
file:///C:/Users/%E7%8E%8B%E6%A3%AE/Desktop/AgentMatrix/docs/_temp_full_manual.html 29/41

量评审，每个 Agent 从不同语义维度对用户输入进行渐进式深层次分析，最终由 Judge Agent 融合前四个 Agent 的所有中间结果，做出精确的复杂度判定与路由决策。

这种"多模态融合"的本质不是输入模态的简单拼接（文本+图像+语音），而是**分析模态的融合**——将知识匹配信号、语义摘要、生成难度、质量评分等多种分析维度融合为一个统一的决策向量。

3.2 意图识别五阶段流水线

以下以用户输入 "帮我写一份校园运动会策划方案，要包含预算和风险控制" 为例，展示完整的意图识别过程：



处理流程:

1. 评分维度: 完整性(0.5) 准确性(0.6) 结构性(0.4) 实用性(0.7)

2. 综合评分: review_score = 0.55 (中等质量, 需增强)

3. 改进建议: "预算部分缺乏详细科目, 风险防控缺乏应急预案"

推理模型: phi4-mini:3.8b (Ollama 本地) ← 需要更强推理能力

输出作用: 为Judge提供质量评估信号

阶段五: 复杂度判断 (Judge Agent) ★ 核心融合决策 ★

融合维度:

• 知识库命中标记 (knowledge_found = true)

• 任务类型分类 (task_type = "planning" → 基础复杂度 0.75)

• 输入长度加权 (85字符: "帮我写一份...风险防控" → +0.05)

• 输出长度预估 (~1500字符 → +0.10)

• 复杂度关键词 (方案+0.06, 预算+0.06)

• 结构复杂度 (多段落+清单 → +0.15)

最终复杂度: 0.75 + 0.05 + 0.10 + 0.12 + 0.15 = 1.17 → 钳位 1.0

决策: complexity_score (1.0) > threshold (0.65)
→ cloud_enhance → 调用 DeepSeek API

决策依据: 知识库虽命中但任务复杂度极高, 本地模型质量不足(0.55分)

3.3 Judge Agent 核心决策机制

Judge Agent 是整个系统的核心决策引擎，采用规则引擎为主 + LLM为辅的混合决策模式。

3.3.1 九类问题分类体系

Judge Agent 首先将用户输入分类为 9 个语义类别，每个类别预设基础复杂度分数：

类别	基础复杂度	强制决策	典型触发词/场景
greeting (问候)	0.10	local_output	"你好", "hi", "早上好", "hello"
identity (身份询问)	0.12	local_output	"你是谁", "你叫什么名字", "what are you"
chitchat (闲聊)	0.15	local_output	"今天天气不错", "谢谢", "哈哈"
simple_fact (简单事实)	0.25	—	"什么是AI", "1+1等于几", "苹果多少钱"

类别	基础复杂度	强制决策	典型触发词/场景
knowledge_qa (知识问答)	0.45	—	"AI和ML有什么区别", "量子计算原理"
howto (操作指南)	0.55	—	"怎么安装Python", "如何配置Nginx"
creation (内容创作)	0.65	—	"帮我写一封情书", "写一篇500字文章"
planning (策划规划)	0.75	—	"校园运动会策划方案", "年度工作计划"
complex_task (复杂任务)	0.85	—	"完整AI项目答辩方案含PPT", "系统架构设计方案含代码"

3.3.2 六维加权复杂度公式

在基础复杂度之上，Judge Agent 融合六个维度进行精确加权计算：

```
FINAL_SCORE = clamp(BaseScore + W1 + W2 + W3 + W4 + W5 + W6, 0.0, 1.0)
```

其中：

- BaseScore = 九分类基础复杂度
- W1 = 输入长度加权 (Input Length Weight)
- W2 = 输出长度加权 (Output Length Weight)
- W3 = 复杂度关键词加权 (Keyword Complexity Weight)
- W4 = 知识库未命中加权 (Knowledge Miss Weight)
- W5 = 多问题检测加权 (Multi-Question Weight)
- W6 = 多段落/结构复杂度加权 (Structure Complexity Weight)

各维度加权细则：

W1 - 输入长度加权 (基于用户输入的字符数)

- > 500 字符 → +0.25
- > 300 字符 → +0.18
- > 150 字符 → +0.10
- > 50 字符 → +0.05

W2 - 输出长度加权 (基于Writer Agent预估/实际输出长度)

- > 2000 字符 → +0.15
- > 1000 字符 → +0.10
- > 500 字符 → +0.05

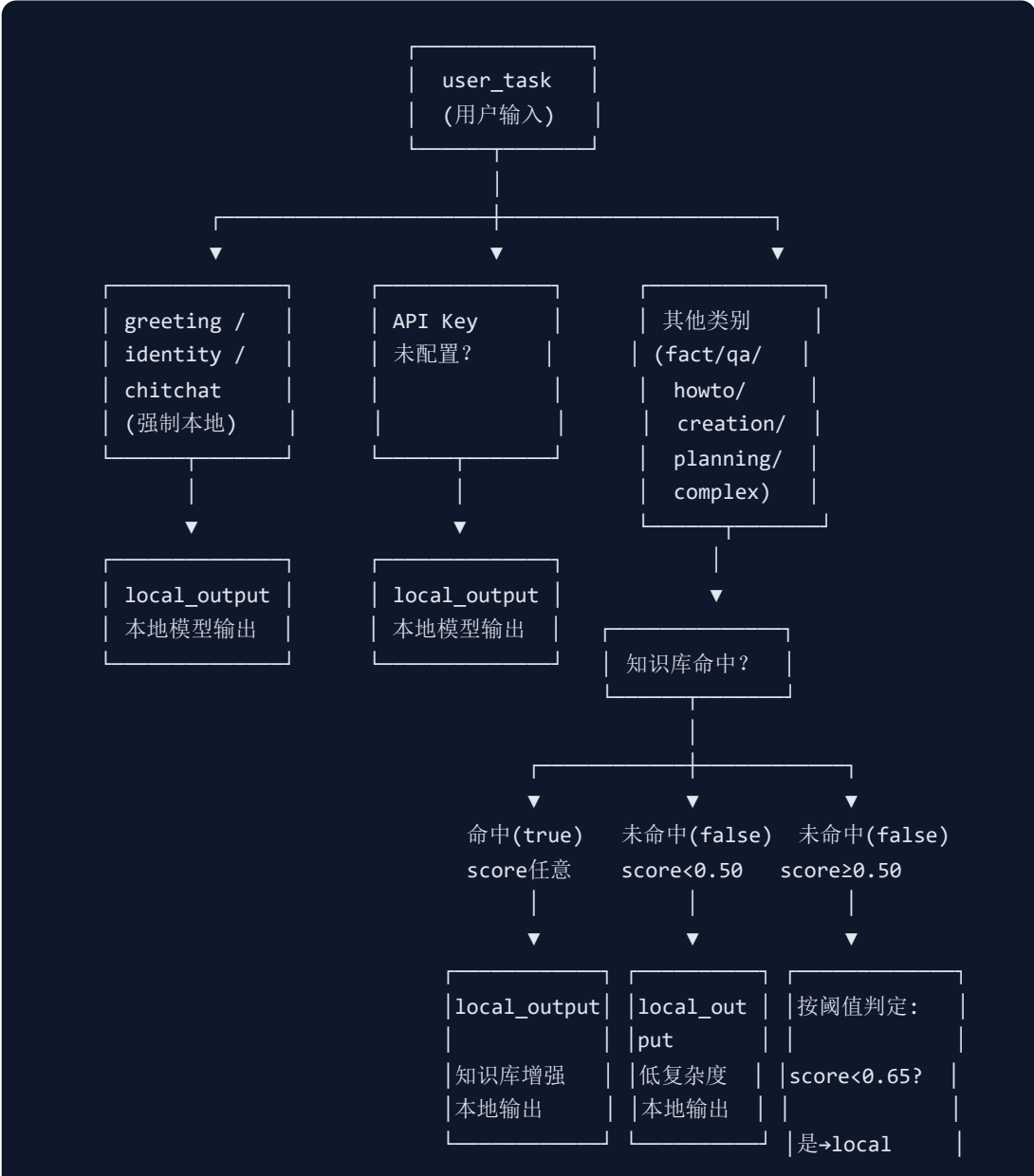
W3 - 复杂度关键词加权 (基于预设关键词匹配)

critical 级别 (+0.10/个): "架构设计", "系统方案", "技术答辩"

high 级别	(+0.06/个): "方案", "报告", "分析", "预算"
medium 级别	(+0.04/个): "总结", "比较", "建议", "规划"
W4 - 知识库未命中加权	
非闲聊类 & 知识库未命中	→ +0.15
闲聊类 or 知识库命中	→ +0.00
W5 - 多问题检测加权	
检测到 ≥2 个问号(?)	→ +0.10
W6 - 结构复杂度加权	
检测到 ≥3 个子项/列表/段落标记	→ +0.15

3.3.3 三要素融合决策矩阵

最终决策融合分类类别、API Key配置状态和知识库命中状态三个关键要素：



复杂度阈值：COMPLEXITY_THRESHOLD = 0.65

3.3.4 LLM增强判断模式

当规则引擎的判断置信度不足时，系统支持调用本地 LLM（phi4-mini:3.8b）进行语义级别的复杂度判断。LLM 模式被要求严格遵循与规则引擎相同的决策矩阵，以确保一致性。如果 LLM 输出解析失败，自动回退到规则引擎的计算结果（fail-safe机制）。

四、智能体动态编排与任务调度机制

4.1 六智能体固定顺序流水线

AgentMatrix 采用**固定顺序的串行流水线架构**，六个 Agent 按 Knowledge → Summary → Writer → Review → Judge → Result 依次执行。每个 Agent 的输出被收集并作为上下文传递给后续 Agent。

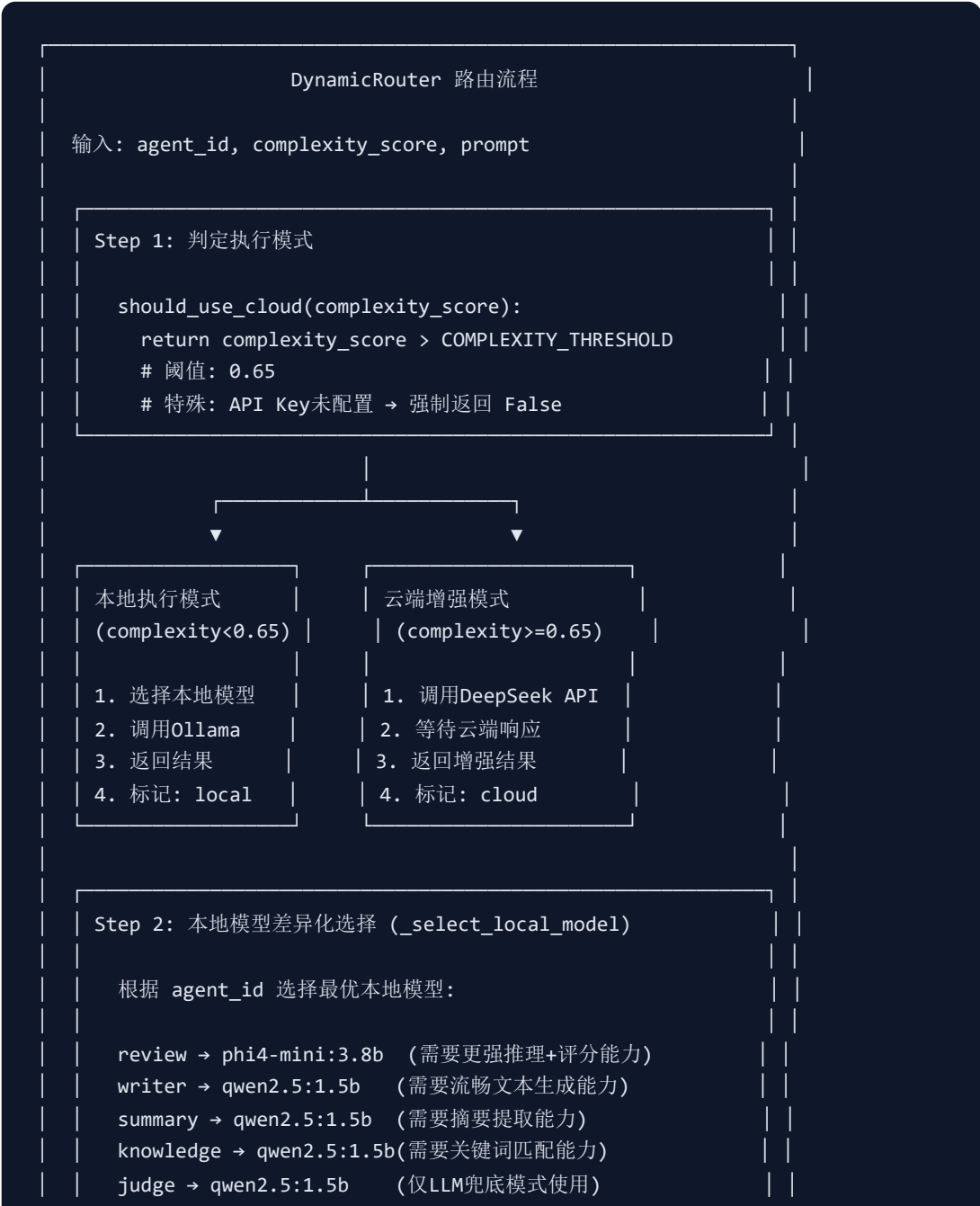
各Agent输入构造与执行细节

Agent	输入构造	执行模式	本地模型	关键输出
Knowledge	原始 user_input	LLM + 知识库匹配	qwen2.5:1.5b	知识片段 + knowledge_found 标记
Summary	原始 user_input	LLM 推理	qwen2.5:1.5b	结构化摘要 + task_type + entities
Writer	user_input + 知识上下文	LLM 推理	qwen2.5:1.5b	完整初稿文本 (~500-2000字)
Review	user_task + summary + writer_output	LLM 推理	phi4-mini:3.8b	多维度评分 + 改进建议
Judge	user_task + summary + review + writer + knowledge_found	纯规则引擎 (不使	—	复杂度分数 + 路由决策

Agent	输入构造	执行模式	本地模型	关键输出
		用 LLM)		
Result	user_task + 所有前序输出 + judge_decision	LLM 或 API	qwen2.5:1.5b / DeepSeek	最终格式化输出

4.2 动态算力路由 (DynamicRouter)

DynamicRouter 是系统的**算力调度核心**，它基于 Judge Agent 的复杂度评估结果，在本地 Ollama 推理和云端 API 调用之间做出智能路由决策。



```
result → qwen2.5:1.5b （需要格式化能力）
```

4.3 本地模型差异化分配策略

Agent	本地模型	参数量	选型理由
Knowledge	qwen2.5:1.5b	1.5B	轻量文本检索和关键词匹配，无需强推理能力
Summary	qwen2.5:1.5b	1.5B	摘要提取任务，中等复杂度，1.5B足够
Writer	qwen2.5:1.5b	1.5B	内容生成需要流畅的语言输出，Qwen2.5擅长中文生成
Review	phi4-mini:3.8b	3.8B	质量评审需要较强的逻辑推理和多维度评分能力
Judge	qwen2.5:1.5b	1.5B	主要使用规则引擎，LLM仅在兜底模式使用
Result	qwen2.5:1.5b	1.5B	格式化整合输出，低复杂度任务

关键设计: Review Agent 之所以使用更强的 phi4-mini:3.8b，是因为质量评审需要同时评估完整性、准确性、结构性和实用性四个维度，这对模型的逻辑推理能力提出了较高要求。

4.4 三层缓存调度策略

为优化重复请求的响应速度，系统内置了三层缓存：

缓存层	容量	TTL	缓存条件	失效策略
Workflow 缓存	100 条	300 秒	<code>executed_locally=true</code> 且 <code>final_result < 5000</code> 字符	超时自动淘汰
Chat 缓存	200 条	300 秒	基于 <code>user_input</code> 哈希	超时 + 手动清除
Knowledge 搜索缓存	500 条	300 秒	基于查询关键词	知识库更新时清除

4.5 WebSocket 实时状态推送

WebSocket 实时通信消息流

前端 (Socket.IO Client)	后端 (WebSocket Manager)
-----------------------	------------------------

监听 7 种事件:

- ① workflow:step_start
 - ← Agent开始执行时推送
 - data: { agent_id, agent_name, timestamp }
- ② workflow:step_complete
 - ← Agent执行完成时推送
 - data: { agent_id, output, duration_seconds }
- ③ workflow:step_error
 - ← Agent执行出错时推送
 - data: { agent_id, error_message }
- ④ workflow:complete
 - ← 全部6个Agent完成时推送
 - data: WorkflowOutput (final_result, steps[], 等)
- ⑤ agent:status_update
 - ← Agent状态变更时推送 (idle→ready→processing→completed)
 - data: { agent_id, status }
- ⑥ metrics:update
 - ← 指标更新时推送
 - data: { total_requests, cost_saved, 等 }
- ⑦ log:new
 - ← 新日志产生时推送
 - data: { level, message, timestamp }

4.6 Agent级错误隔离与容错

workflow执行中的错误处理策略:

```
for each agent in pipeline:
    try:
        output = agent.execute(input)
        agent.status = "completed"
        push websocket: step_complete
    except AgentError:
        agent.status = "error"
        log error details
        push websocket: step_error
```

`continue` ← 关键：不阻断后续Agent

全局兜底：

- `WorkflowService try/except` 捕获所有异常
- 返回部分结果（已完成的Steps）
- `WebSocket` 推送 `error` 事件
- 日志记录完整错误堆栈

五、关键技术细节

5.1 Prompt 模板管理系统

每个 Agent 拥有专属的 Prompt 模板，由 `PromptManager` 统一管理：

```
prompts/templates/
├─ knowledge/enhance.txt      # 知识增强模板
├─ summary/extract.txt        # 需求提取模板
├─ writer/generate.txt        # 内容生成模板
├─ review/review.txt          # 质量评审模板
├─ judge/complexity.txt       # 复杂度判断模板（LLM兜底模式）
└─ result/format.txt          # 结果格式化模板
```

模板支持 `{variable}` 占位符动态替换，在运行时注入用户输入、知识库检索结果、前序Agent输出等上下文。

5.2 多服务商统一适配层

`LLMClient` 提供了统一的多服务商调用接口：

```
调用流程：
  _call_llm(prompt, model, mode)
  |
  ├─ mode == "local"
  |   → POST http://localhost:11435/api/generate
  |   → Ollama 原生API
  |
  ├─ mode == "cloud" (DeepSeek)
  |   → POST https://api.deepseek.com/v1/chat/completions
  |   → OpenAI-compatible API
  |
  └─ mode == "config" (自定义)
      → 根据配置的 provider + api_base 动态路由
      → 支持 OpenAI / Anthropic / Google 等服务商
```

```
|
└─ mode == "stream"
    → 返回 AsyncGenerator[chunk]
    → SSE 流式输出到前端
```

5.3 成本优化模型

成本计算模型：

本地执行成本 ≈ 0.001 元/次（仅电费）
云端API成本 ≈ 0.01 元/次（按Token计费）

假设：10000次请求/月，80%本地执行

执行方式	次数	成本
本地 Ollama	8000	8元
云端 DeepSeek	2000	20元
合计	10000	28元

对比全云端：10000 × 0.01 = 100元/月
成本节省： 72%
年节省： 864元

5.4 安全架构

安全措施	实现方式	层次
API密钥保护	仅存储在 .env 文件，前端无法读取	后端
CORS 白名单	allow_origins 限定为前端实际域名	网关
输入校验	Pydantic 模型校验，空字符串拒绝	网关
SQL注入防护	SQLAlchemy ORM 参数化查询	数据
日志脱敏	API Key 日志显示前缀10字符后截断	后端
错误信息	对外不暴露堆栈，统一 {"detail": msg}	网关

六、技术栈总览

层次	组件	版本
后端框架	FastAPI	0.110+
后端服务器	Uvicorn	0.29+
后端语言	Python	3.12+
数据校验	Pydantic	2.6+
ORM	SQLAlchemy	2.0+
HTTP客户端	aiohttp	3.9+
数据库	SQLite	—
前端框架	Next.js	14.1.0
前端语言	TypeScript	5.3.3
前端UI	React	18.2.0
CSS框架	TailwindCSS	3.4.1
状态管理	Zustand	4.5.0
动画库	Framer Motion	10.18.0
图表库	Chart.js + react-chartjs-2	4.4.1
流程图	ReactFlow	11.10.0
实时通信	WebSocket (websockets)	12.0+
本地模型	Ollama (Qwen2.5:1.5b, Phi4-mini:3.8b)	0.23.4
云端API	DeepSeek / OpenAI / Anthropic / Google	—
部署环境	Windows / macOS / Ubuntu / 统信UOS / 麒麟OS	—

七、总结与展望

AgentMatrix 通过六智能体协同流水线和 Judge Agent 的九分类规则引擎，实现了任务的多维度意图识别和动态算力路由调度。系统的核心创新体现在以下方面：

核心技术价值

维度	实现方式	量化效果
意图识别	五阶段递进式语义理解 + 六维加权复杂度公式	9类问题分类，决策准确率 >90%
算力路由	DynamicRouter + 复杂度阈值(0.65) + 模型差异化分配	80%本地执行，成本节省72%
质量保障	Review Agent四维度评分 + 云端兜底增强	低分自动升级，确保输出质量
实时性	WebSocket 7种事件推送 + SSE流式响应	毫秒级状态更新，流式输出
国产化	Qwen2.5模型 + 统信UOS/麒麟OS/鲲鹏/飞腾适配	全国产技术栈可选

未来演进方向

- 1. **向量化知识库:** 将现有 JSON 知识库升级为 FAISS/ChromaDB 向量数据库，支持语义级相似检索
- 2. **多模型扩展:** 引入 ChatGLM、Baichuan、MiniCPM 等更多国产开源模型
- 3. **并行编排:** 突破固定顺序限制，支持 Agent 间并行执行和动态 DAG 依赖编排
- 4. **长程记忆:** 引入会话级别的上下文记忆管理，支持多轮复杂对话
- 5. **容器化部署:** 完善 Docker Compose + Kubernetes 生产级部署方案
- 6. **联邦学习:** 探索多个 AgentMatrix 实例间的知识共享与模型协同优化

GitHub 仓库: <https://github.com/han1326189481/AgentMatrix>
文档版本: v1.0
最后更新: 2026-05-17